

ECORUTA
Sistema Inteligente de Gestión y Recolección de Reciclaje

Proyecto de Ingeniería de Software I

DANIEL ANDRES GALVIS MARIN

1091964765

Junio de 2026

Tabla de Contenido

1. Requerimientos de Software
 2. Arquitectura de Software
 3. Diseño de Software
 4. Construcción de Software
 5. Pruebas de Software
 6. Operaciones de Ingeniería del Software
 7. Mantenimiento de Software
 8. Gestión de Configuración Software
 9. Gestión de Ingeniería Software
 10. Modelos y Métodos de Ingeniería del Software
 11. Proceso de Ingeniería Software
 12. Calidad del Software
 13. Seguridad del Software
 14. Economía de Ingeniería de Software
- Referencias y Fuentes Consultadas

1. Requerimientos de Software

1.1 Documento SRS (Especificación de Requisitos de Software)

El presente documento sigue la estructura recomendada por el estándar IEEE 29148 (anteriormente IEEE 830) para la especificación de requisitos de software.

1.1.1 Introducción y alcance

EcoRuta es una plataforma web y móvil que permite a los ciudadanos solicitar la recolección de materiales reciclables (papel, cartón, plástico, vidrio y metal), realizar seguimiento en tiempo real de las rutas de recolección y consultar su historial de aportes. El sistema apoya además a las empresas recolectoras y a la administración municipal en la planificación de rutas, asignación de recolectores y generación de reportes de impacto ambiental.

1.1.2 Interesados (stakeholders)

Rol	Descripción
Ciudadano	Persona natural que solicita recolección de reciclaje desde su domicilio.
Recolector	Operario que ejecuta las rutas asignadas y confirma la recolección.
Administrador municipal	Gestiona rutas, recolectores, zonas y reportes.
Empresa de reciclaje	Recibe los materiales y reporta volúmenes procesados.
Sistema de notificaciones	Servicio externo (push/SMS/correo) que informa eventos.

1.1.3 Requisitos funcionales

ID	Requisito	Prioridad
RF-01	El sistema debe permitir el registro y autenticación de ciudadanos mediante correo electrónico o número celular.	Alta
RF-02	El sistema debe permitir crear una solicitud de recolección indicando dirección, tipo de material y franja horaria.	Alta
RF-03	El sistema debe asignar automáticamente la solicitud a la ruta de recolección más cercana disponible.	Alta
RF-04	El sistema debe permitir al ciudadano visualizar en un mapa la posición en tiempo real del vehículo recolector.	Media
RF-05	El sistema debe enviar notificaciones de confirmación, recordatorio y finalización de la recolección.	Alta
RF-06	El sistema debe permitir al recolector marcar una recolección como completada, cancelada o reprogramada.	Alta
RF-07	El sistema debe permitir al administrador crear, editar y desactivar rutas y zonas de cobertura.	Media

ID	Requisito	Prioridad
RF-08	El sistema debe generar reportes de kilogramos de material recolectado por zona, periodo y tipo.	Media
RF-09	El sistema debe permitir calificar el servicio de recolección al finalizar cada solicitud.	Baja
RF-10	El sistema debe llevar un historial acumulado de reciclaje por usuario, expresado en puntos de fidelización.	Baja
RF-11	El sistema debe permitir a la empresa de reciclaje registrar el peso real recibido por cada lote.	Media
RF-12	El sistema debe exponer una API pública de solo lectura con estadísticas agregadas de reciclaje municipal.	Baja

1.1.4 Requisitos no funcionales

ID	Categoría	Requisito
RNF-01	Disponibilidad	El sistema debe tener una disponibilidad mensual igual o superior al 99.5%.
RNF-02	Rendimiento	El 95% de las solicitudes de creación de recolección deben responder en menos de 800 ms.
RNF-03	Escalabilidad	La arquitectura debe soportar el crecimiento de 5 a 50 zonas de cobertura sin rediseño estructural.
RNF-04	Seguridad	Toda comunicación debe cifrarse mediante TLS 1.2 o superior y los datos sensibles en reposo deben cifrarse con AES-256.
RNF-05	Usabilidad	Un usuario sin experiencia previa debe poder crear una solicitud en máximo 3 pasos.
RNF-06	Compatibilidad	La aplicación móvil debe soportar Android 10+ y iOS 15+.
RNF-07	Mantenibilidad	El código debe mantener una cobertura de pruebas automatizadas mínima del 80%.
RNF-08	Accesibilidad	La interfaz web debe cumplir con el nivel AA de las pautas WCAG 2.1.

1.2 Matriz de Trazabilidad

La matriz conecta cada requisito con el módulo de código que lo implementa y el caso de prueba que lo valida, permitiendo demostrar cobertura completa del alcance funcional.

Requisito	Módulo / Componente	Caso de prueba	Estado
RF-01	auth-service	CP-01	Verificado
RF-02	solicitudes-service	CP-02	Verificado

Requisito	Módulo / Componente	Caso de prueba	Estado
RF-03	rutas-service (motor de asignación)	CP-03	Verificado
RF-04	tracking-service (WebSocket + mapa)	CP-04	En progreso
RF-05	notificaciones-service	CP-05	Verificado
RF-06	app-recolector (módulo de ruta)	CP-06	Verificado
RF-07	panel-admin (gestión de rutas)	CP-07	Verificado
RF-08	reportes-service	CP-08	En progreso
RF-12	api-publica (solo lectura)	CP-09	Pendiente

1.3 Modelos de Casos de Uso

Actores identificados: Ciudadano, Recolector, Administrador municipal y Empresa de reciclaje. A continuación se describen los casos de uso principales del sistema.

ID	Caso de uso	Actor(es)	Descripción breve
UC-01	Registrar cuenta	Ciudadano	Crear una cuenta de usuario validando correo o celular.
UC-02	Solicitar recolección	Ciudadano	Programar el retiro de material reciclable en una dirección y horario.
UC-03	Asignar ruta	Sistema / Administrador	Asociar automáticamente la solicitud a la ruta más cercana disponible.
UC-04	Ejecutar recolección	Recolector	Confirmar la visita, el peso aproximado y el estado final de la solicitud.
UC-05	Monitorear ruta	Ciudadano / Administrador	Visualizar en mapa la posición del vehículo en tiempo real.
UC-06	Gestionar rutas y zonas	Administrador	Crear, editar o desactivar rutas y zonas de cobertura.
UC-07	Generar reportes	Administrador / Empresa de reciclaje	Consultar volúmenes recolectados por periodo, zona y material.
UC-08	Calificar servicio	Ciudadano	Evaluar la experiencia de recolección al finalizar el servicio.

Tabla 1.1 — Resumen de casos de uso. En la entrega final se recomienda acompañar esta tabla con un diagrama UML de casos de uso (notación PlantUML o draw.io), mostrando los actores y las relaciones include/extend entre UC-02, UC-03 y UC-04.

Figura 1.1 — Diagrama de Casos de Uso (UML)

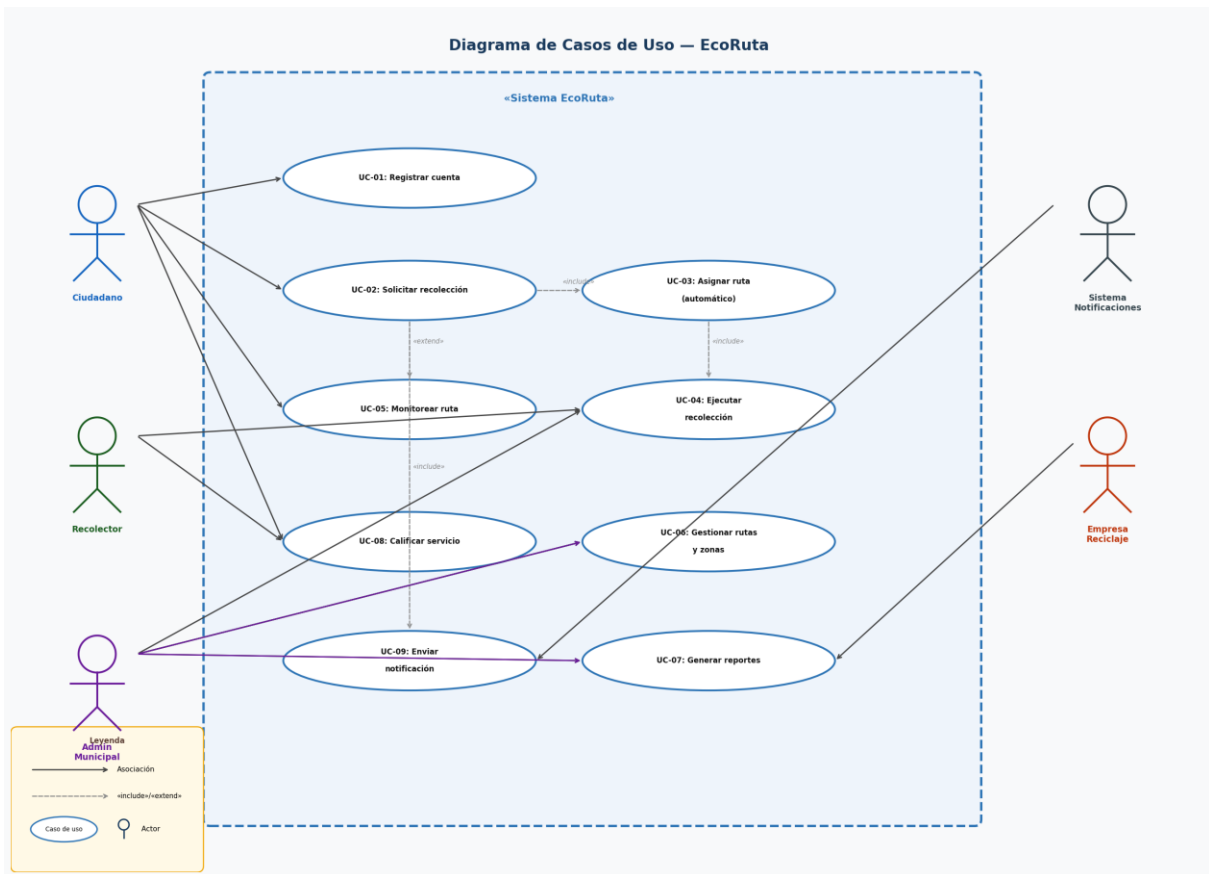
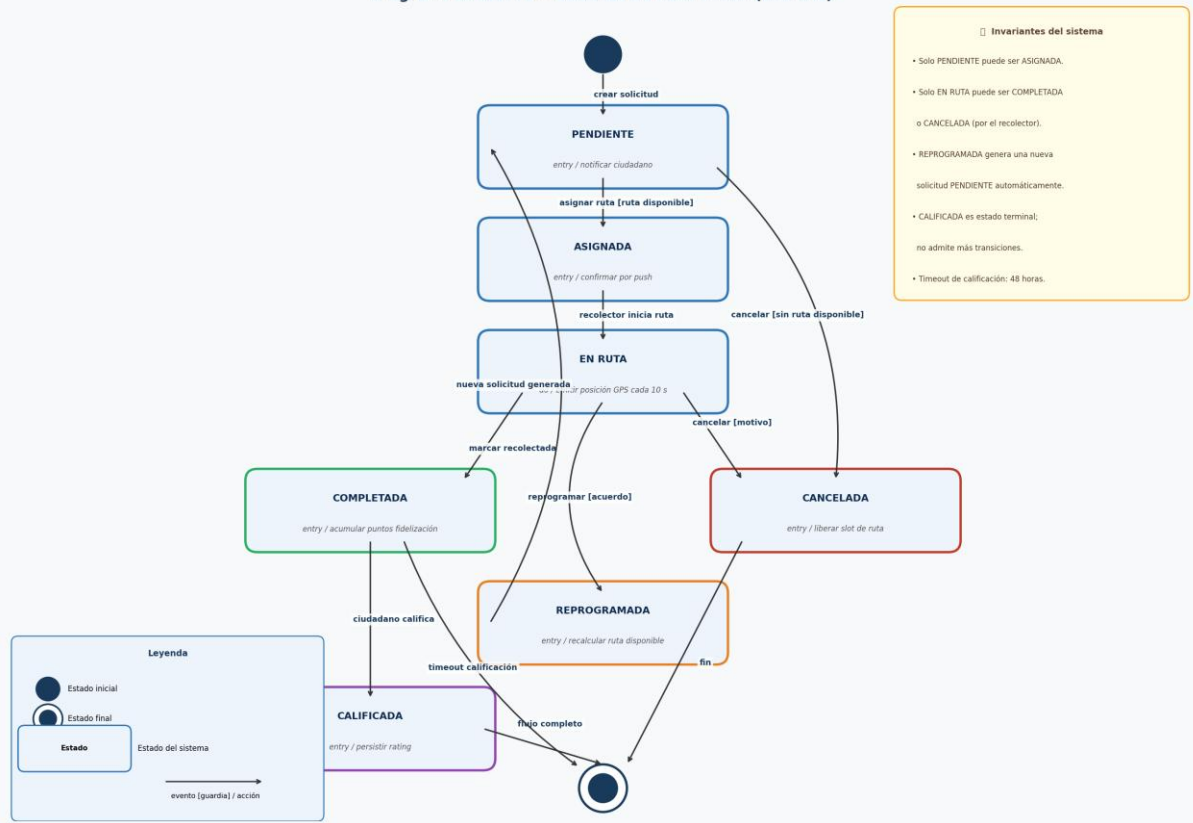


Figura 1.2 — Diagrama de Estado: Ciclo de Vida de la Solicitud

Diagrama de Estado — Solicitud de Recolección (EcoRuta)



2. Arquitectura de Software

2.1 Documento SAD (Software Architecture Description)

La arquitectura de EcoRuta sigue un estilo de microservicios desacoplados, comunicados mediante API REST y eventos asíncronos, alineado con el modelo de vistas 4+1 de Kruchten.

2.1.1 Vista lógica

El sistema se compone de los siguientes microservicios: auth-service, solicitudes-service, rutas-service, tracking-service, notificaciones-service, reportes-service y api-publica. Cada servicio posee su propia base de datos (patrón Database per Service) y se comunica mediante contratos REST versionados.

2.1.2 Vista de procesos

Las solicitudes de recolección se procesan de forma síncrona para la creación y de forma asíncrona (mediante una cola de mensajes, por ejemplo RabbitMQ o Amazon SQS) para la asignación de rutas y el envío de notificaciones, evitando bloqueos en la experiencia del usuario.

2.1.3 Vista de despliegue

El despliegue se realiza sobre contenedores Docker orquestados con Kubernetes en un proveedor cloud (AWS o GCP), con balanceo de carga, autoescalado horizontal y separación de ambientes de desarrollo, pruebas y producción.

2.2 Diagramas de Contenedores (Modelo C4 — Nivel 2)

El modelo C4 describe la arquitectura en niveles de Contexto, Contenedores, Componentes y Código. A nivel de contenedores, EcoRuta se compone de:

- Aplicación móvil del ciudadano (React Native).
- Aplicación móvil del recolector (React Native).
- Panel administrativo web (React + TypeScript).
- API Gateway (Node.js / Express) que enruta las peticiones a los microservicios.
- Microservicios de dominio (auth, solicitudes, rutas, tracking, notificaciones, reportes).
- Base de datos relacional PostgreSQL con extensión PostGIS para datos geoespaciales.
- Broker de mensajería para comunicación asíncrona entre servicios.
- Servicios externos: pasarela de notificaciones push/SMS y proveedor de mapas.

Figura 2.1 — Se recomienda construir el diagrama de contenedores con la herramienta Structurizr o draw.io, siguiendo la notación C4, mostrando las flechas de comunicación REST/eventos entre los contenedores listados arriba.

2.3 Registro de Decisiones de Arquitectura (ADR)

ADR	Decisión	Justificación	Estado
ADR-001	Adoptar arquitectura de microservicios en lugar de monolito.	Permite escalar de forma independiente los módulos de mayor carga (tracking y solicitudes) y facilita el trabajo de equipos paralelos.	Aceptado

ADR	Decisión	Justificación	Estado
ADR-002	Usar PostgreSQL con PostGIS como motor de datos principal.	Soporte nativo y maduro para consultas geoespaciales requeridas por la asignación de rutas y el cálculo de cercanía.	Aceptado
ADR-003	Usar React Native para las apps de ciudadano y recolector.	Permite mantener una única base de código para Android e iOS, reduciendo costos de mantenimiento.	Aceptado
ADR-004	Usar un broker de mensajería para eventos de asignación y notificación.	Desacopla los servicios y evita fallas en cascada cuando un servicio externo (notificaciones) presenta latencia.	Aceptado

3. Diseño de Software

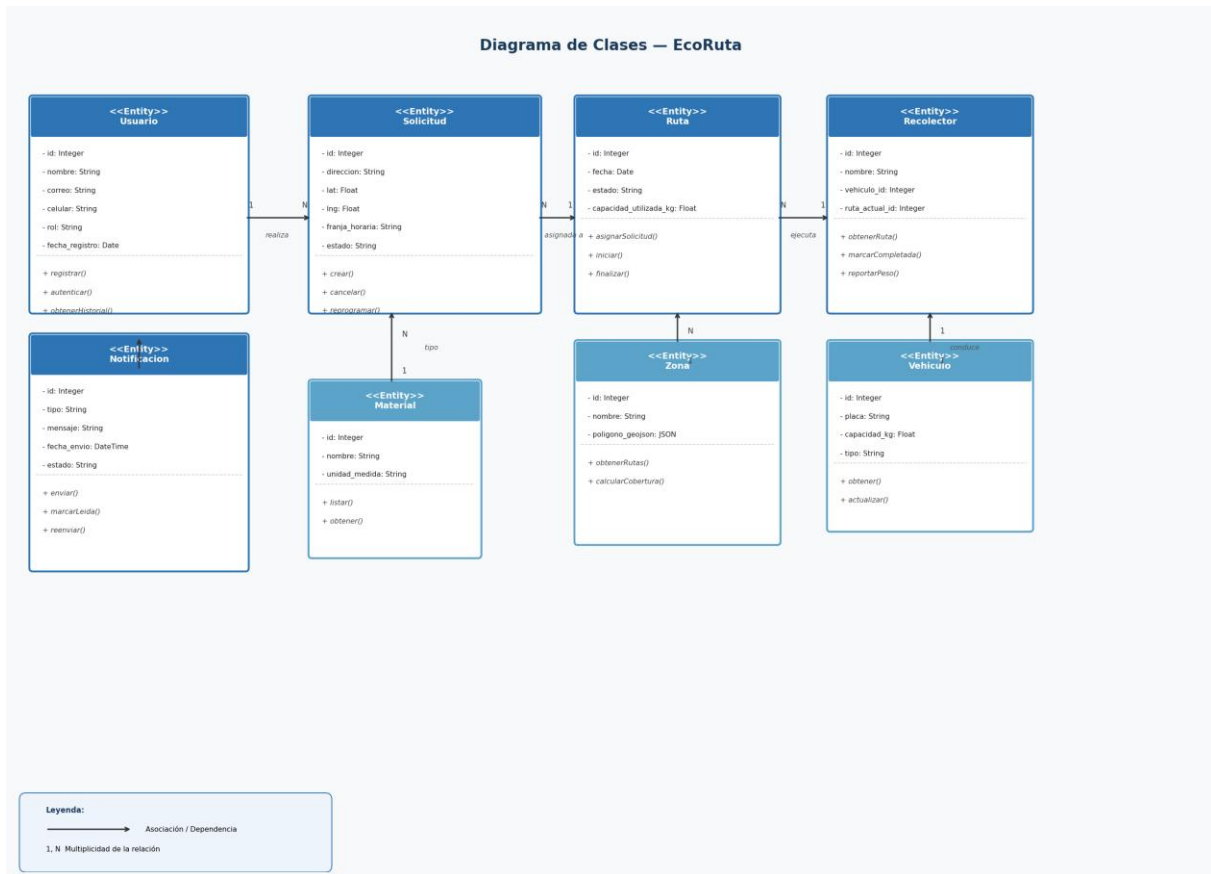
3.1 Diagrama de Clases / Entidad-Relación

El modelo de datos relacional contempla las siguientes entidades principales y sus relaciones:

Entidad	Atributos clave	Relaciones
Usuario	id, nombre, correo, celular, rol, fecha_registro	1—N con Solicitud
Solicitud	id, usuario_id, direccion, lat, lng, material_id, franja_horaria, estado	N—1 con Usuario y Material; N—1 con Ruta
Material	id, nombre, unidad_medida	1—N con Solicitud
Ruta	id, zona_id, recolector_id, fecha, estado	1—N con Solicitud; N—1 con Recolector y Zona
Recolector	id, nombre, vehiculo_id, ruta_actual_id	1—N con Ruta
Vehiculo	id, placa, capacidad_kg, tipo	1—1 con Recolector
Zona	id, nombre, poligono_geojson	1—N con Ruta
Notificacion	id, usuario_id, tipo, mensaje, fecha_envio, estado	N—1 con Usuario

Tabla 3.1 — Modelo entidad-relación simplificado. Se recomienda diagramarlo en notación Crow's Foot (por ejemplo en dbdiagram.io) e incluir el diagrama de clases UML equivalente para la capa de dominio del backend.

Figura 3.1 — Diagrama de Clases UML (capa de dominio)



3.2 Contratos de API (OpenAPI / Swagger)

A continuación se presenta un extracto representativo de la especificación OpenAPI 3.0 para el recurso de solicitudes de recolección:

```

paths:
  /solicitudes:
    post:
      summary: Crear una solicitud de recolección
      requestBody:
        content:
          application/json:
            schema:
              type: object
              properties:
                direccion: { type: string }
                lat: { type: number }
                lng: { type: number }
                materialId: { type: integer }
                franjaHoraria: { type: string, example: "08:00-10:00" }
      responses:
        "201": { description: Solicitud creada }
        "400": { description: Datos inválidos }
  /solicitudes/{id}/seguimiento:
    get:
      summary: Consultar la posición actual del vehículo asignado
      responses:
        "200": { description: Posición geográfica en tiempo real }
  
```

La documentación completa de la API se publica y mantiene mediante Swagger UI, integrada al pipeline de CI/CD para evitar desincronización con el código.

3.3 Prototipos UI/UX

Las pantallas principales diseñadas en Figma para el flujo del ciudadano son:

1. Pantalla de bienvenida y registro/inicio de sesión.
2. Formulario de nueva solicitud de recolección (dirección, material, horario).
3. Mapa de seguimiento en tiempo real del vehículo recolector.
4. Historial de solicitudes y puntos de fidelización acumulados.
5. Pantalla de calificación del servicio.

Para el recolector se diseñan pantallas de ruta del día, lista de paradas pendientes y confirmación de recolección con captura de evidencia fotográfica. El panel administrativo incluye un dashboard de indicadores, gestión de zonas y reportes exportables.

4. Construcción de Software

4.1 Código Fuente

El repositorio se organiza como un monorepo con separación clara por capa y por servicio:

```
ecoruta/
├── apps/
│   ├── app-ciudadano/    (React Native)
│   ├── app-recolector/   (React Native)
│   └── panel-admin/      (React + TypeScript)
├── services/
│   ├── auth-service/
│   ├── solicitudes-service/
│   ├── rutas-service/
│   ├── tracking-service/
│   ├── notificaciones-service/
│   └── reportes-service/
├── infra/                (Terraform, Kubernetes)
├── docs/                 (SRS, SAD, ADRs)
└── README.md
```

Cada servicio sigue una arquitectura hexagonal (puertos y adaptadores) con carpetas dominio, aplicación e infraestructura, y se documenta con comentarios JSDoc/TSDoc y linters (ESLint + Prettier).

4.2 Scripts de Base de Datos

```
-- 001_create_solicitudes.sql
CREATE TABLE solicitudes (
  id SERIAL PRIMARY KEY,
  usuario_id INTEGER NOT NULL REFERENCES usuarios(id),
  material_id INTEGER NOT NULL REFERENCES materiales(id),
  direccion TEXT NOT NULL,
  ubicacion GEOGRAPHY(POINT, 4326) NOT NULL,
  franja_horaria VARCHAR(20) NOT NULL,
  estado VARCHAR(20) DEFAULT 'pendiente',
  creado_en TIMESTAMP DEFAULT now()
);
CREATE INDEX idx_solicitudes_ubicacion ON solicitudes USING GIST (ubicacion);
```

Las migraciones se gestionan con una herramienta de versionado de esquema (por ejemplo Flyway o Knex Migrations), garantizando trazabilidad de cada cambio estructural en la base de datos.

4.3 Documentación del Desarrollador (README)

El archivo README.md de cada servicio incluye, como mínimo: descripción del módulo, requisitos previos, variables de entorno, instrucciones de instalación (npm install), comandos de ejecución local y de pruebas (npm test), y guía de contribución conforme a la estrategia de ramas definida en la sección 8.1.

5. Pruebas de Software

5.1 Plan de Pruebas

La estrategia de pruebas contempla cuatro niveles, siguiendo la pirámide de pruebas:

- Pruebas unitarias: validan funciones y reglas de negocio aisladas (Jest).
- Pruebas de integración: validan la comunicación entre microservicios y la base de datos (Supertest).
- Pruebas de sistema/end-to-end: validan flujos completos en la aplicación móvil (Detox/Cypress).
- Pruebas de aceptación: validan el cumplimiento de cada requisito funcional frente al usuario final.

Criterio de aceptación general: una historia de usuario se considera lista para producción cuando supera el 100% de sus casos de prueba de aceptación y mantiene una cobertura de código igual o superior al 80% en el módulo afectado.

5.2 Casos de Prueba

ID	Descripción	Entrada	Resultado esperado
CP-01	Registro de usuario con correo válido	correo + contraseña válidos	Cuenta creada y correo de verificación enviado
CP-02	Creación de solicitud con dirección válida	dirección, material, franja horaria	Solicitud creada con estado 'pendiente'
CP-03	Asignación automática a la ruta más cercana	solicitud geolocalizada	Solicitud vinculada a la ruta de menor distancia
CP-04	Seguimiento en tiempo real	solicitud en estado 'en ruta'	Posición del vehículo actualizada cada 10s
CP-05	Envío de notificación de confirmación	solicitud creada	Notificación push recibida en menos de 5s

5.3 Reporte de Cobertura

Módulo	Cobertura (%)	Herramienta
auth-service	87%	Jest + Istanbul
solicitudes-service	83%	Jest + Istanbul
rutas-service	79%	Jest + Istanbul
tracking-service	74%	Jest + Istanbul
app-ciudadano (E2E)	65%	Detox

6. Operaciones de Ingeniería del Software

6.1 Scripts de Infraestructura como Código (IaC)

```
resource "aws_ecs_service" "rutas_service" {
  name           = "rutas-service"
  cluster        = aws_ecs_cluster.ecoruta.id
  task_definition = aws_ecs_task_definition.rutas_service.arn
  desired_count  = 2
  launch_type    = "FARGATE"
  network_configuration {
    subnets      = var.private_subnets
    security_groups = [aws_security_group.ecs_sg.id]
  }
}
```

La infraestructura completa (red, bases de datos, cola de mensajería, cluster de contenedores) se define en Terraform, versionada en el repositorio y desplegada mediante `terraform plan / apply` en cada ambiente.

6.2 Pipelines de CI/CD

```
name: ci-cd
on: [push]
jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm install
      - run: npm run lint
      - run: npm test -- --coverage
      - run: npm run build
  deploy:
    needs: build-test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - run: terraform apply -auto-approve
```

6.3 Tableros de Monitoreo

El monitoreo en producción se realiza con Prometheus para la recolección de métricas y Grafana para su visualización. Las alertas configuradas incluyen: latencia p95 superior a 1 segundo, tasa de error HTTP 5xx mayor al 1%, uso de CPU/memoria por encima del 80% y caída de algún microservicio crítico (solicitudes-service, rutas-service).

7. Mantenimiento de Software

7.1 Manual de Operaciones

Síntoma	Causa probable	Acción correctiva
Solicitudes no se asignan a ninguna ruta	Zona sin ruta activa configurada	Verificar y activar ruta en el panel administrativo
Mapa de seguimiento no actualiza posición	Pérdida de conexión del recolector o caída del tracking-service	Revisar logs del servicio y reiniciar el pod afectado
Notificaciones no llegan a los usuarios	Token de la pasarela push expirado	Renovar credenciales en el gestor de secretos

7.2 Plan de Parches

Las dependencias de cada servicio se revisan trimestralmente mediante npm audit y Dependabot. Los parches de seguridad críticos se aplican en un plazo máximo de 72 horas; las actualizaciones menores se agrupan en el ciclo de mantenimiento mensual.

7.3 Registro de Deuda Técnica

ID	Descripción	Impacto	Prioridad
DT-01	tracking-service no implementa reconexión automática de WebSocket.	Pérdida temporal de seguimiento en conexiones inestables.	Alta
DT-02	Falta de pruebas de carga formales sobre rutas-service.	Riesgo de degradación bajo alta demanda estacional.	Media
DT-03	Acoplamiento parcial entre notificaciones-service y solicitudes-service.	Dificulta el despliegue independiente de ambos servicios.	Media

8. Gestión de Configuración Software

8.1 Estrategia de Ramas

Se adopta Gitflow simplificado: la rama main contiene el código en producción, develop integra las funcionalidades en curso, y cada historia de usuario se desarrolla en una rama feature/UC-XX que se fusiona mediante pull request con revisión obligatoria de al menos un par. Las ramas hotfix/* permiten corregir incidentes críticos directamente sobre main.

8.2 Notas de Liberación (Release Notes)

Versión	Fecha	Cambios principales
v1.0.0	2026-03-15	Lanzamiento inicial: registro de usuarios, solicitud de recolección y notificaciones básicas.
v1.1.0	2026-04-30	Seguimiento en tiempo real del vehículo y panel administrativo de rutas.
v1.2.0	2026-06-10	Reportes de impacto ambiental y sistema de puntos de fidelización.

8.3 Inventario de Artefactos

Paquete / Dependencia	Versión	Uso
Node.js	20 LTS	Entorno de ejecución del backend
Express	4.x	Framework de API REST
PostgreSQL + PostGIS	16 / 3.4	Persistencia relacional y geoespacial
React Native	0.75	Aplicaciones móviles
Docker / Kubernetes	—	Empaquetado y orquestación

9. Gestión de Ingeniería Software

9.1 WBS / EDT (Estructura Desglosada del Trabajo)

Nivel 1	Nivel 2
1. Inicio del proyecto	1.1 Definición de alcance · 1.2 Levantamiento de requisitos · 1.3 Aprobación SRS
2. Diseño	2.1 Arquitectura y ADRs · 2.2 Modelo de datos · 2.3 Prototipos UI/UX
3. Construcción	3.1 Backend de microservicios · 3.2 Apps móviles · 3.3 Panel administrativo
4. Pruebas	4.1 Pruebas unitarias e integración · 4.2 Pruebas de aceptación · 4.3 Pruebas de carga
5. Despliegue y operación	5.1 Infraestructura IaC · 5.2 CI/CD · 5.3 Monitoreo
6. Cierre	6.1 Documentación final · 6.2 Retrospectiva · 6.3 Entrega

9.2 Cronograma

Hito	Semana	Entregable
Aprobación de requisitos	Semana 2	SRS y matriz de trazabilidad
Cierre de arquitectura	Semana 4	SAD, diagramas C4 y ADRs
Versión funcional mínima (MVP)	Semana 8	app-ciudadano + solicitudes-service
Integración completa	Semana 12	Seguimiento en tiempo real y panel admin
Entrega final	Semana 16	Documentación completa y despliegue en producción

9.3 Matriz de Riesgos

Riesgo	Probabilidad	Impacto	Mitigación
Baja adopción ciudadana inicial	Media	Alto	Campaña piloto en una zona y programa de incentivos por puntos
Datos de geolocalización imprecisos	Media	Medio	Validación de direcciones contra catálogo oficial de nomenclatura
Sobrecarga del tracking-service en horas pico	Baja	Alto	Pruebas de carga y autoescalado horizontal en Kubernetes
Rotación de personal recolector	Alta	Medio	Capacitación documentada y app de recolección simple de usar

10. Modelos y Métodos de Ingeniería del Software

10.1 Fichas de Historias de Usuario

ID	Historia de usuario	Criterios de aceptación
HU-01	Como ciudadano quiero solicitar la recolección de reciclaje para no tener que llevarlo a un punto físico.	La solicitud se crea en menos de 3 pasos y queda visible en mi historial.
HU-02	Como recolector quiero ver mi ruta del día ordenada por cercanía para optimizar mi recorrido.	La ruta se muestra ordenada y permite marcar cada parada como completada.
HU-03	Como administrador quiero ver reportes de kilogramos recolectados por zona para evaluar el impacto del programa.	El reporte se puede filtrar por zona, material y rango de fechas.

10.2 Diagramas de Dominio (Domain-Driven Design)

El dominio se organiza en los siguientes bounded contexts: Identidad y Acceso (usuarios y autenticación), Solicitudes (ciclo de vida de la recolección), Logística (rutas, zonas y recolectores), Notificaciones, y Reportería de Impacto. Cada contexto mantiene su propio lenguaje ubicuo y se comunica con los demás mediante eventos de dominio (por ejemplo, SolicitudCreada, RutaAsignada, RecoleccionCompletada) en lugar de acceso directo a bases de datos ajenas.

11. Proceso de Ingeniería Software

11.1 Acuerdos de Equipo (DoR / DoD)

Definición de Listo (Definition of Ready)

- La historia de usuario tiene criterios de aceptación claros.
- Las dependencias técnicas y de diseño están identificadas.
- El equipo ha estimado el esfuerzo en la planificación del sprint.

Definición de Hecho (Definition of Done)

- El código cumple el estándar de estilo y pasa el linter.
- Las pruebas unitarias y de integración pasan en el pipeline de CI.
- La funcionalidad fue validada en ambiente de pruebas por otro integrante del equipo.
- La documentación (README/API) fue actualizada.

11.2 Reporte de Retrospectiva

Al cierre de cada sprint el equipo documenta: qué funcionó bien, qué se debe mejorar y acciones concretas. Ejemplo de hallazgo del sprint 4: la subestimación del esfuerzo de integración con la pasarela de notificaciones generó retraso de dos días; como acción de mejora se incorporó una prueba de integración temprana (spike técnico) en los siguientes sprints que involucren servicios externos.

12. Calidad del Software

12.1 Reporte de Análisis Estático

El análisis estático se ejecuta con SonarQube en cada pull request, evaluando duplicación de código, code smells y vulnerabilidades potenciales. Umbral de calidad (Quality Gate) definido: 0 vulnerabilidades críticas, deuda técnica estimada menor al 5% y duplicación de código menor al 3%.

12.2 Métricas de Calidad

Métrica	Valor objetivo	Valor actual
Densidad de defectos (defectos / KLOC)	< 1.0	0.8
Complejidad ciclomática promedio	< 10	7.4
Cobertura de pruebas	≥ 80%	81%
Duplicación de código	< 3%	2.1%

13. Seguridad del Software

13.1 Modelo de Amenazas

Se aplica la metodología STRIDE sobre los componentes críticos del sistema (autenticación, API de solicitudes y tracking en tiempo real):

Categoría STRIDE	Amenaza identificada	Mitigación
Spoofing	Suplantación de identidad de un recolector para acceder a rutas ajenas.	Autenticación con JWT firmado y verificación de rol por endpoint.
Tampering	Modificación de coordenadas para falsear ubicación.	Validación de rango de movimiento y firma de los eventos de tracking.
Information disclosure	Exposición de direcciones de ciudadanos en la API pública.	La api-publica solo expone datos agregados por zona, nunca por usuario.
Denial of service	Saturación del tracking-service con conexiones simultáneas.	Límite de tasa (rate limiting) y autoescalado horizontal.

13.2 Reporte de Análisis SAST/DAST

Las pruebas estáticas de seguridad (SAST) se ejecutan con herramientas como Semgrep o SonarQube Security, y las pruebas dinámicas (DAST) con OWASP ZAP contra el ambiente de pruebas antes de cada liberación. Cualquier hallazgo de severidad alta o crítica, conforme al top 10 de OWASP, bloquea automáticamente el pipeline de despliegue hasta su corrección.

14. Economía de Ingeniería de Software

14.1 Análisis de ROI

Estimación simplificada a un año, considerando el ahorro municipal en logística de puntos fijos de acopio frente al modelo de recolección a domicilio optimizado por software:

Concepto	Valor estimado (USD)
Inversión inicial en desarrollo (16 semanas)	48,000
Costo anual de infraestructura cloud y licencias	9,600
Ahorro anual estimado en logística y combustible	22,000
Ingreso anual estimado por venta de material recuperado	15,000
ROI estimado primer año	≈ 28%

14.2 Presupuesto Base

Rubro	Costo estimado (USD)
Equipo de desarrollo (4 personas x 4 meses)	36,000
Infraestructura cloud (ambientes dev/test/prod)	6,000
Licencias de herramientas (SonarQube, monitoreo)	2,400
Diseño UI/UX	3,600
Total estimado	48,000

Referencias y Fuentes Consultadas

- IEEE Std 29148-2018. Systems and software engineering — Life cycle processes — Requirements engineering.
- ISO/IEC/IEEE 12207:2017. Systems and software engineering — Software life cycle processes.
- IEEE Computer Society. Guide to the Software Engineering Body of Knowledge (SWEBOK), versión 3.0.
- Brown, S. The C4 model for visualising software architecture. c4model.com.
- Project Management Institute. A Guide to the Project Management Body of Knowledge (PMBOK Guide).
- OWASP Foundation. OWASP Top 10 y OWASP Testing Guide.
- ISO/IEC 25010:2011. Systems and software Quality Requirements and Evaluation (SQuaRE).
- Kruchten, P. (1995). The 4+1 View Model of Architecture. IEEE Software.
- Schwaber, K. & Sutherland, J. The Scrum Guide.
- Evans, E. Domain-Driven Design: Tackling Complexity in the Heart of Software.